

최종 결과 보고서

(2024학년도 1학기)

과제명	요트 다이스 보드게임 구현		
과목 /담당교수	프로그래밍설계 / 정복래교수님		
제출일자	2024. 06. 17		
연구참여자 (담당분야)	팀장 : 이현영 (각 기능에 맞는 사용자 정의함수 작성, 과제총괄 및 보고서 작성) 팀원 : 강현욱 (각 기능에 맞는 사용자 정의함수 작성, 게임의 주요 알고리즘 구현) 팀원 : 김승호 (각 기능에 맞는 사용자 정의함수 작성, 코드 최적화) 팀원 : 이윤상 (각 기능에 맞는 사용자 정의함수 작성, 테스트 및 디버깅)		
팀명	5분 대기조	팀 번호	5

목 차

1. 설계과제 제목	3
2. 연구목적	3
3. 설계과제의 필요성	3
4. 설계과제의 목표	3
5. 설계과정	4
6. 제작	11
7. 시험	15
8. 평가	18
9. 추진체계	20
10. 설계 추진일정	21
11. 결론	22

1. 설계과제의 제목 : 요트 다이스 보드게임 구현

2. 연구목적

- 1) 알고리즘 개발 및 최적화
- 2) 사용자 인터페이스 설계
- 3) 점수보드 구현 및 통계적 분석 기능 설계
- 4) 다중플레이어 게임 구현

3. 설계과제의 필요성

- 1) 요트다이스 게임의 주사위 던지기 및 점수 계산 기능을 위해 알고리즘을 효율적으로 설계하고 최적화하는 기술이 필요하다.
- 2) 실제 사용자가 사용하기 쉽도록 사용자 친화적인 인터페이스를 디자인하고 구현하는 기술이 필요하다.
- 3) 각각의 차례에서 굴린 주사위의 합을 분석하여 예상 점수를 출력, 원하는 카테고리 선택 후 해당 카테고리의 점수판에 저장하여 출력하는 기능을 구현해야한다.
- 4) 플레이어 수와 이름 입력 후 각각의 차례에서 주사위를 굴리고 점수를 계산하여 승자를 결정하는 다중 플레이어로 즐길 수 있는 게임을 구현해야한다.

4. 설계과제의 목표

- 1) 프로그램의 구현 : 요트다이스 보드게임을 c언어로 구현
- 2) 인터페이스 구현 : 사용자 친화적 인터페이스 구현
- 3) 모듈화 : 각 기능을 모듈화하여 유지보수가 용이한 코드 구현
- 4) 구현하고자 하는 기능 : 턴 제 다중플레이어 게임, 예상점수 출력 등 기능적 요구사항 명세서의 내용

표 1. 본 프로젝트의 현실적 제한요소 항목

현실적 제한 요소들	내 용 (Content)
경제	-게임의 구현에 필요한 메모리 등을 효율적으로 활용 -최적화된 코드 작성으로 자원의 낭비 최소화
편리	- 사용자의 쉬운 게임접근을 위한 인터페이스 제공 - 명확한 게임설명 안내 제공
윤리	- 사행성 목적으로 사용되지 않도록 함
사회	- 여러 사람과 즐길 수 있는 게임 구현으로 화합 도모

5. 설계과정

5.1 설계 기초이론

5.1.1 데이터 구조 : 배열을 사용하여 주사위 값, 점수 등을 효율적으로 저장하고 관리하였으며, 점수 계산에 필요한 정보를 손쉽게 얻을 수 있게 하였다.

5.1.2 프로그래밍 기법 : 주사위를 굴리는 과정 등에서 반복문을 이해하고 활용하고, 특정 조건에 따라 턴 변경, 점수 계산을 하는 조건문을 이해하고 활용하였다.

5.1.3 알고리즘 : 주사위의 결과를 토대로 점수를 계산할 때 효율적인 알고리즘을 위해 선택정렬, 해당하는 카테고리를 찾기 위한 탐색 등과 같은 기본 알고리즘을 이해하고 활용하여 각 기능을 구현할 효율적인 알고리즘을 설계하였다.

5.1.4 사용자 인터페이스 : 터미널을 통해 점수판 등 게임상태와 점수를 출력하고 사용자의 입력을 받아 게임을 진행하도록 하였다.

5.2 소프트웨어 기능블록도 (개념설계)

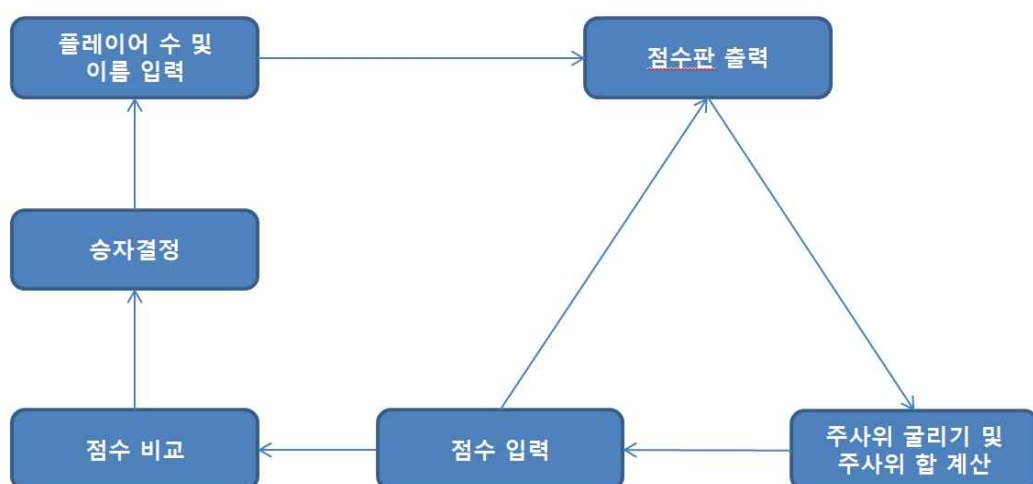


그림 5.2.1 '요트다이스'의 기능 블록도

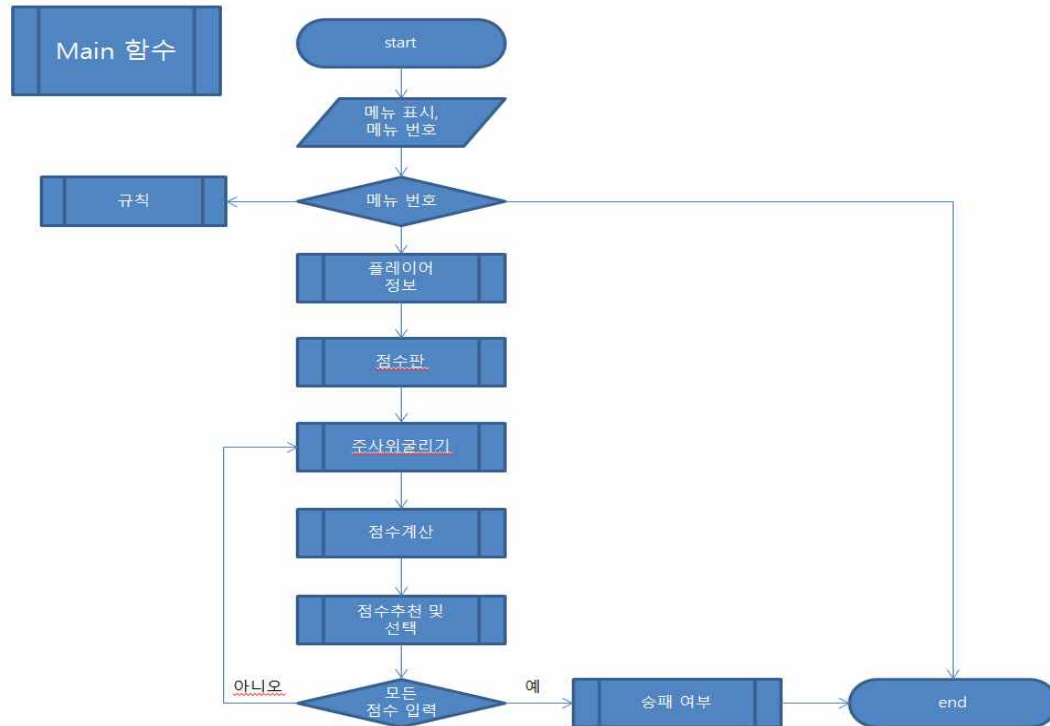
5.3 기능적 요구사항 명세서

ID	요구사항	내용	설명	우선 순위
R_01	점수 계산	주사위 결과를 바탕으로 점수를 계산한다.	다양한 점수 카테고리(스트레이트, 포카드 등) 별로 점수를 계산할 수 있어야 한다. 그 후 계산된 점수로 승자를 결정한다.	3
R_02	점수판 출력	입력된 결과를 점수판으로 출력한다.	입력된 플레이어의 이름과 점수 카테고리 별로 점수가 입력된 점수판을 모든 플레이어가 볼 수 있게 출력한다.	5
R_03	점수 선택	자신의 주사위 결과를 자신이 원하는 점수 카테고리에 입력한다.	본인 차례에 굴려 나온 주사위 결과를 12가지의 점수 카테고리 중에서 플레이어가 선택하는 곳에 점수를 입력한다.	4
R_04	점수 추천	점수판에 주사위 결과를 바탕으로 선택하기 좋은 카테고리들을 표시한다.	주사위 결과를 바탕으로 각 카테고리에 입력 시 1점 이상의 예상 점수를 플레이어가 보기 쉽도록 표시해준다.	10
R_05	다중 플레이어 구현	다중 플레이어 게임을 구현한다.	최소 2명 최대 4명의 플레이어 수와 입력 이름을 입력받아 입력된 인원수를 토대로 점수판 크기를 결정하고 생성한다.	6
R_06	주사위 굴리기	5개의 주사위를 굴린다.	5개의 주사위를 생성하고 사용자가 세 번까지 굴릴 수 있도록 하며 굴림 사이에 원하는 주사위를 선택하여 고정할 수 있도록 한다.	1
R_07	규칙 설명	사용자에게 게임 시작 전 게임에 대한 규칙을 제공한다.	게임 진행 방식과 점수 체계에 대한 설명을 제공한다. 그리고 언제든지 게임 중 규칙을 볼 수 있도록 한다.	9
R_08	승패 여부	다중 플레이어간 승패 여부를 확인한다	점수가 가장 높은 플레이어의 이름을 출력한다.	11
R_09	턴 변경	플레이어가 행동을 마치면 턴을 변경한다.	입력된 플레이어의 순서대로 턴을 진행하고, 플레이어가 입력할 점수 카테고리를 선택 완료하면 상대방의 차례로 턴을 변경한다.	7
R_10	게임 메뉴 인터페이스	게임 메뉴 인터페이스를 제공한다.	게임 시작, 게임 규칙, 게임 종료, 전적 등의 인터페이스를 제공한다.	8
R_11	주사위 정렬	주사위를 오름차순으로 정렬한다	선택정렬을 사용하여 주사위를 오름차순으로 정렬한다.	2

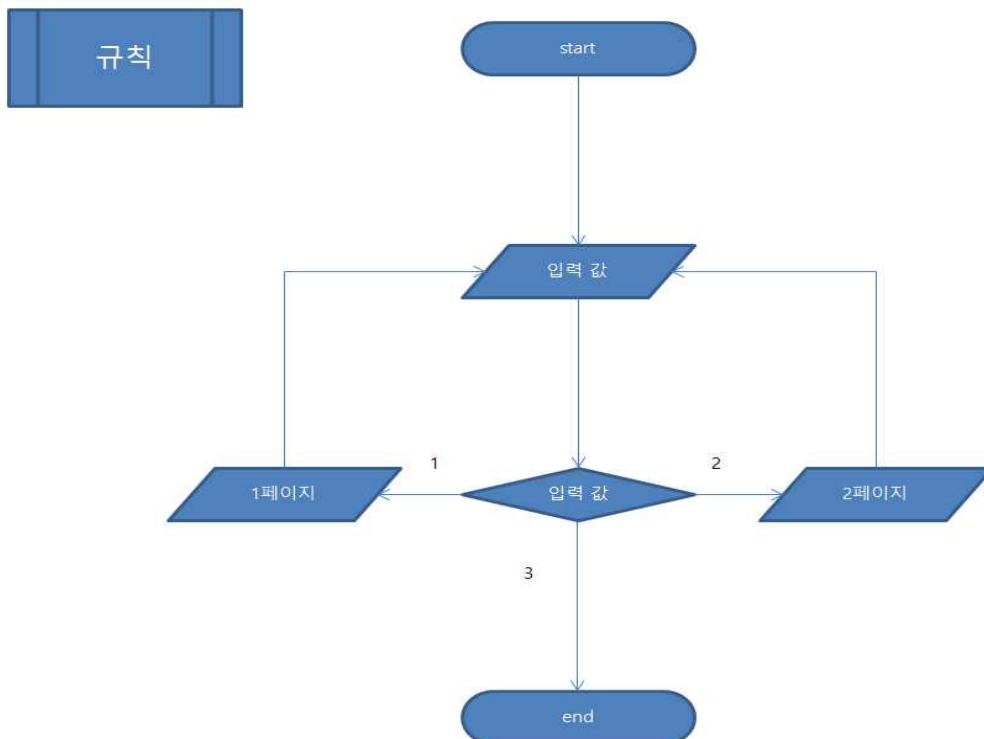
5.4 전체/부분 순서도 (상세설계)

순서도

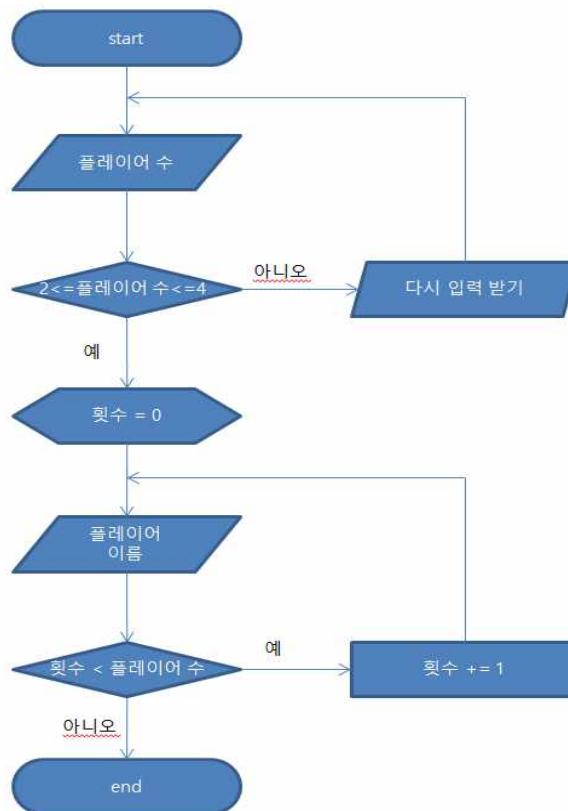
a) 메인함수



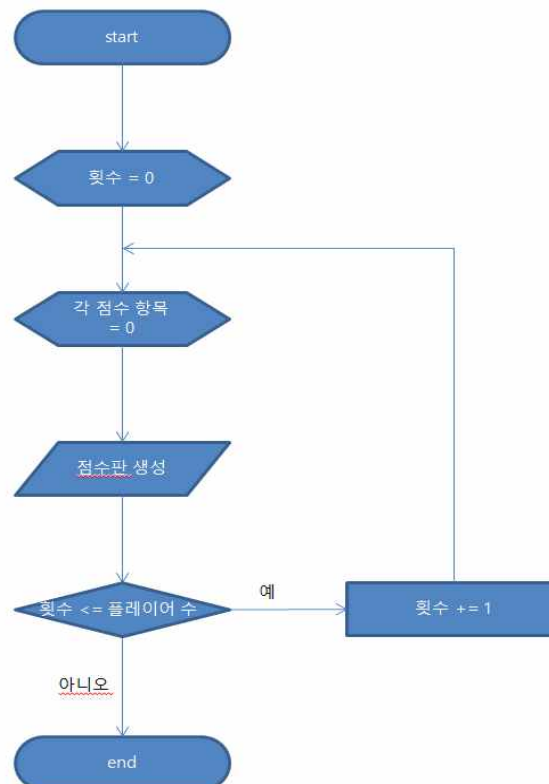
b) 규칙 모듈



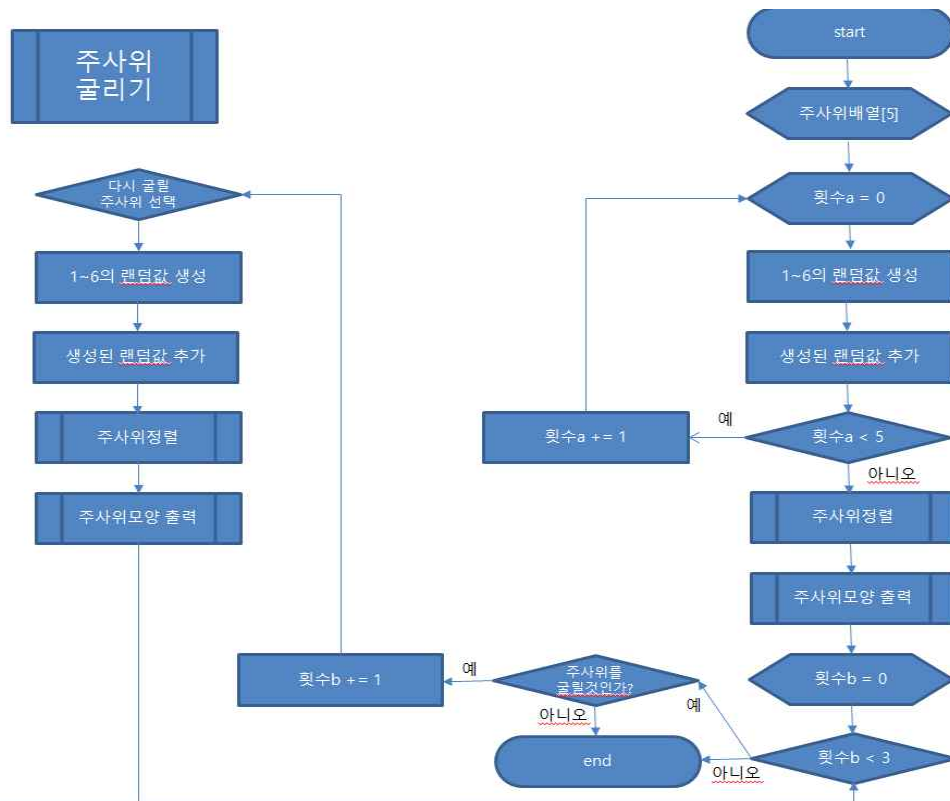
c) 플레이어 정보 모듈



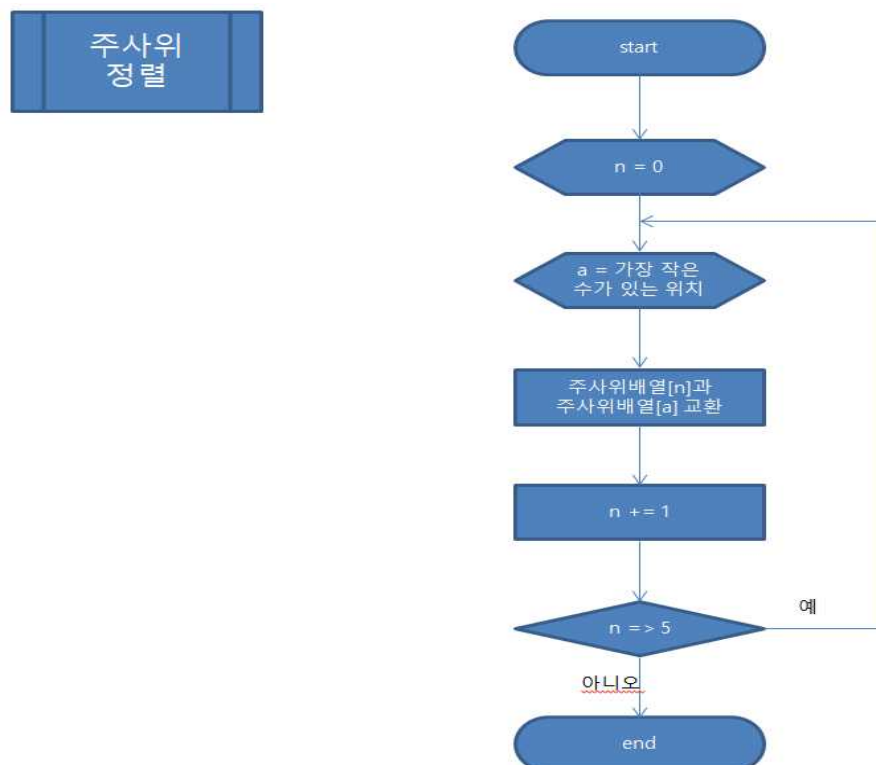
d) 점수판 모듈



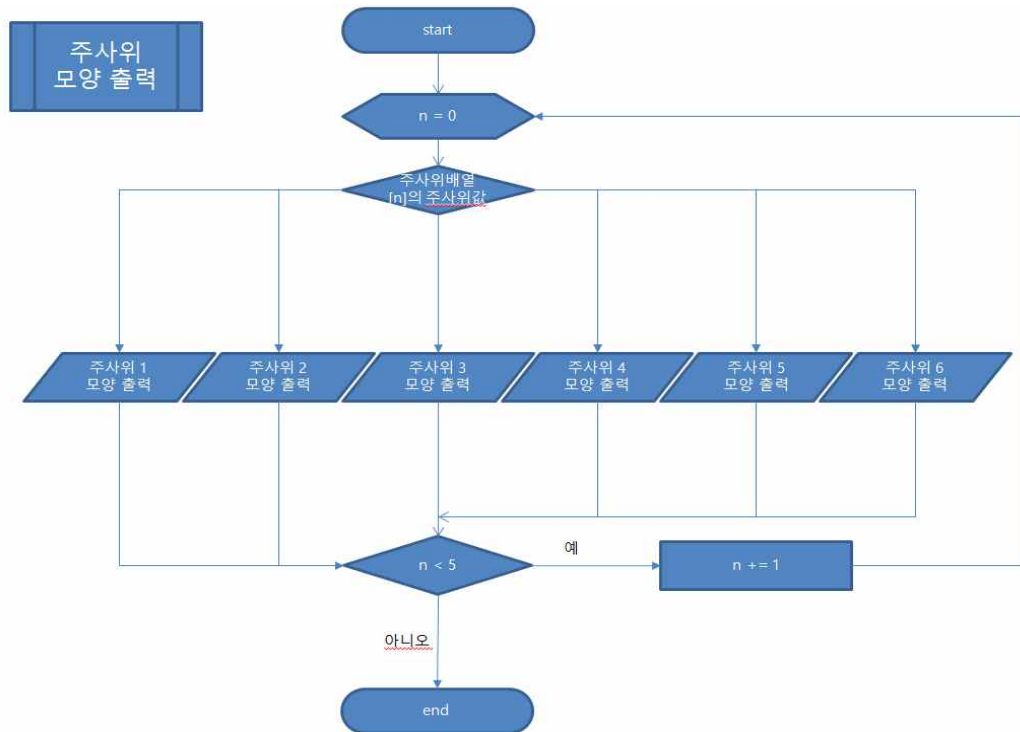
e) 주사위 굴리기 모듈



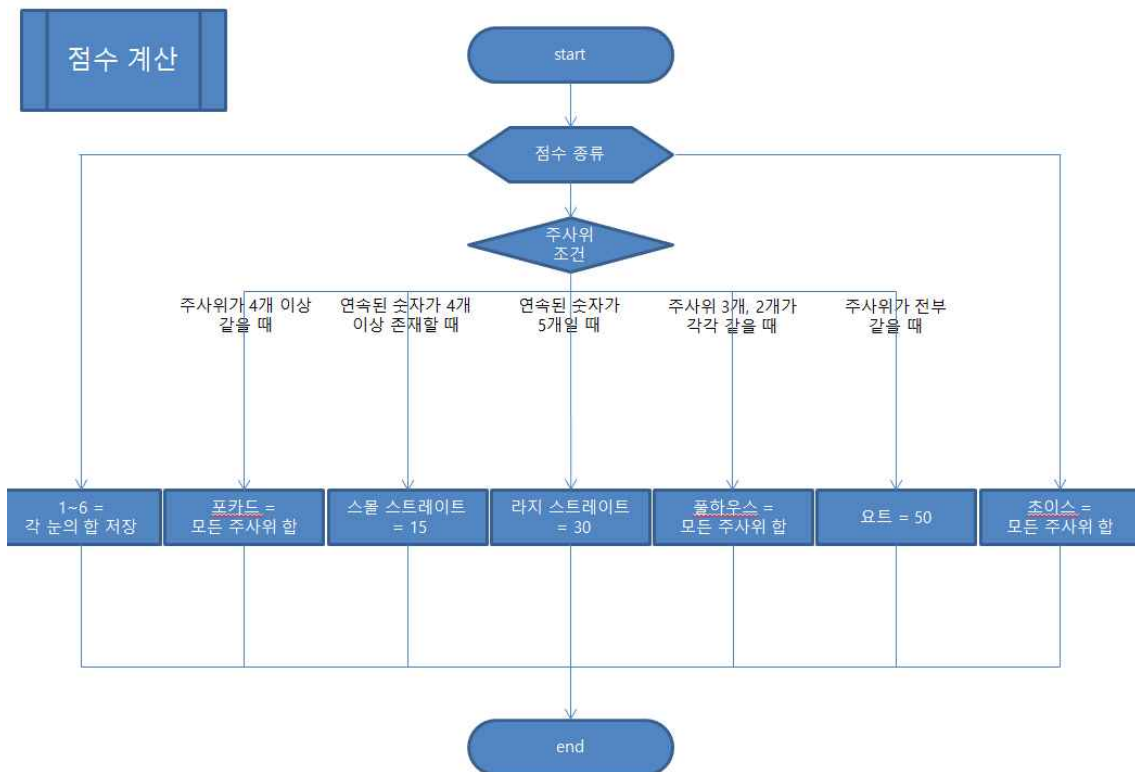
f) 주사위 정렬 모듈



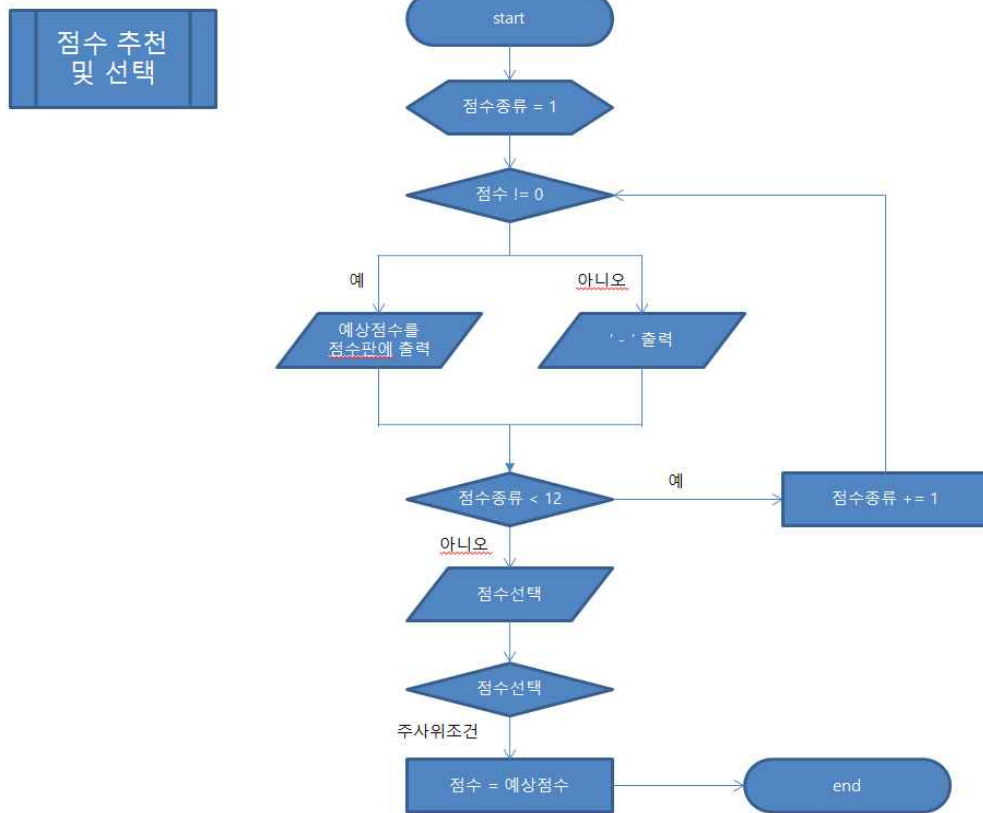
g) 주사위 모양 출력 모듈



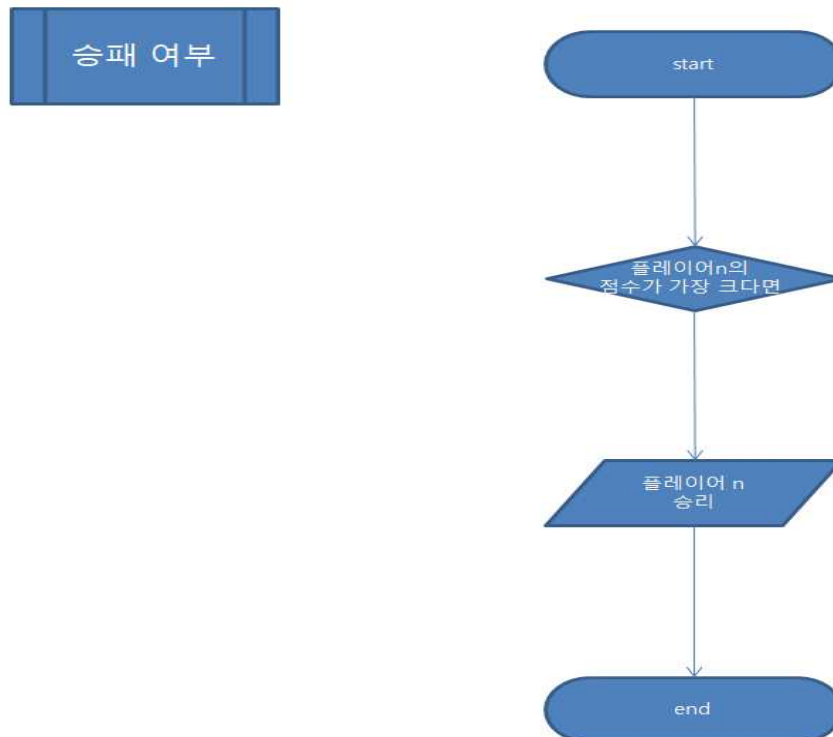
h) 점수 계산 모듈



i) 점수추천 및 선택 모듈



j) 승패 여부 모듈



6. 제작(Implementation)

6.1 제작과정

6.1.1 플레이어 정보 입력 함수

1) 아스키 코드를 사용하여 영문과 한글을 구분해서 길이를 증가시켜 따로 알 수 있게 하였다.

```
for (int i = 0; i < len; i++) {
    if ((nickname[i] >= 0x00 && nickname[i] <= 0x7F)) { //아스키코드를 사용하여 영문과 한글 구분
        english++; //영어 길이 카운트 증가
    }
    else {
        korean++; //한글 길이 카운트 증가
    }
}
return (english <= 8 && korean <= 8);
```

2) strcpy를 써서 4명 미만일 경우 빈 배열에 '-'를 복사해 넣었다.

```
for (i = playerCount; i < MAX_PLAYERS; i++) { //4인 미만일 경우 빈 닉네임은 '-'로 채움
    strcpy(nickname[i], "-");
}
```

6.1.2 규칙 함수

- 스위치문을 사용하여 입력받은 수에 맞는 기능을 실행하였다.

```
switch (choice)
{
    case 1:
        FileContent("rule1.txt");
        break;
    case 2:
        FileContent("rule2.txt");
        break;
    case 3:
        printf("프로그램을 종료합니다.\n");
        return;
    default:
        printf("잘못된 선택입니다. 다시 선택하세요.\n");
        break;
}
```

6.1.3 점수선택 함수

1) scount라는 배열을 통해 각 플레이어의 해당 카테고리가 채워졌는지 여부를 판단하고 채워지지 않았을 경우에만 실행한다.

```
void selectscore(int tcount) { //점수를 선택하는 함수
    int category = 0;
    int scoreType = 1;
    extern int x; //x는 메인함수에 있는 함수로 현재 몇 번째 플레이어인지 판단하는 변수
    extern int scount[4][12]; //플레이어 별로 해당 카테고리가 채워졌는지 여부 판단을 위한 배열
    while (1) { //유효한 카테고리 값을 입력할 때까지 반복
        ask;
        printf("원하는 카테고리를 입력하십시오(1 ~ 12) : ");
        scanf_s("%d", &category);
        getchar();
        if (scount[x][category - 1] == 1) { //해당 인덱스가 1이라면 해당 카테고리는 채워져있음
            ask;
            printf("이미 입력한 점수입니다 다시 선택해주세요");
            getchar();
        }
        else if (category < 1 || category > 12) {
            ask;
            printf("1~12사이의 숫자를 입력해주세요");
            getchar();
        }
        else
            break;
    }
}
```

2) x라는 현재 차례인 플레이어가 누군지 판단하는 변수로 스위치문을 사용하여 첫 번째 플레이어 차례라면 첫 번째 플레이어가 사용하는 점수 배열과 midTotal, Total을 사용한다. 이때 midTotal이 63점이 넘으면 보너스 35점을 추가하며, t1~t4함수를 사용하여 보너스 점수가 한 번이라도 추가 됐다면 더 이상 추가하지 않도록 한다.

```
switch (x + 1) { // 턴 별로 점수가 입력되는 플레이어 결정
    case 1:
        p1score[category - 1] = score[category - 1]; //선택한 카테고리에 해당하는 첫 번째 플레이어의 점수배열에 계산된 점수 저장
        if (category <= 6) { // 선택한 카테고리가 6이하면 midTotal에도 합산
            midTotal[0] += p1score[category - 1];
        }
        Total[0] += p1score[category - 1]; //7이상이면 Total에만 합산
        if (midTotal[0] >= 63 && t1 == 0) { //midTotal이 63점 이상이면 보너스 35점 추가
            Total[0] += 35;
            t1 = 1; //보너스 점수는 한 번만 추가하기 위한 함수
        }
        scount[0][category - 1] = 1; //해당 카테고리는 사용하였다는 의미로 1추가
        break;
}
```

6.1.4 주사위 굴리기 함수

- 다른 함수에서 입력받은 다시 굴릴 주사위 수만큼 다시 굴리고 싶은 주사위를 사용자에게 입력받아 주사위를 다시 굴리는 함수이다. 유효한 값을 입력 받으면 reroll_indice의 해당 값의 인덱스에 1을 저장하고 1이 된 인덱스에 해당하는 주사위만 값을 다시 설정한다. 유효하지 않은 값은 invalid_input 플래그를 설정하여 입력 버퍼를 지운 후 다시 입력 받도록 하였다.

```
while (1) {
    ask;
    printf("다시 굴릴 주사위의 번호를 선택하세요 (1-5, 공백으로 구분): ");
    int dices_entered = 0;
    int invalid_input = 0;
    for (int i = 0; i < reroll; i++) { //위에서 입력받은 다시 굴릴 주사위 개수만큼 반복
        int index;
        if (scanf_s("%d", &index) != 1 || index < 1 || index > 5 || reroll_indice[index - 1]) { //유효성 검사
            if (index < 1 || index > 5) { //1에서 5사이인지 확인
                ask;
                printf("잘못된 번호입니다. 1에서 5 사이의 숫자를 입력하세요.\n");
            }
            else if (reroll_indice[index - 1]) { //중복된 번호인지 확인
                ask;
                printf("이미 선택된 번호입니다. 다른 번호를 선택하세요.\n");
            }
            invalid_input = 1; //유효하지 않은 입력으로 플래그 설정
            while (getchar() != '\n'); // 잘못된 입력 버퍼 비우기
            break;
        }
        reroll_indice[index - 1] = 1; //유효한 입력일 경우 인덱스 설정
        dices_entered++; //바꾼 주사위 수 증가
    }
    if (!invalid_input && dices_entered == reroll) { //유효한 입력이 모두 완료되었는지 확인
        break;
    }
    else {
        ask;
        printf("잘못된 입력입니다. %d개의 유효한 번호를 입력하세요.\n", reroll);
        while (getchar() != '\n');
        for (int i = 0; i < 5; i++) {
            reroll_indice[i] = 0; // 재입력을 위해 배열 초기화
        }
    }
}
```

6.1.5 점수계산 함수

- 점수 족보에 따른 조건을 통해 주사위 값을 점수 배열에 저장한다. count에는 주사위 눈의 크기 별로 개수를 저장하고, temp[0]은 같은 주사위가 2개 있을 때 1 저장, temp[1]은 3개 있을 때 1 저장, temp[2]는 연속된 숫자의 개수를 확인하여 저장한다. 그 후 각 점수 조건에 맞게 계산하여 저장한다.

```

int* score_cal(int dice[]) {
    int count[6] = { 0 }; // 주사위 눈의 개수를 세기 위한 배열
    int temp[3] = { 0 };
    for (int i = 0; i < 6; i++) { // 1~6페어와 초이스 점수 초기화
        score[i] = 0;
    }

    for (int i = 0; i < 5; i++) { // 1~6 페어 (점수 족보 배열의 0번부터 5번까지)
        score[dice[i] - 1] += dice[i];
        score[6] += dice[i]; // 초이스 (점수 족보 배열의 6번)
        count[dice[i] - 1]++;
    }

    for (int i = 0; i < 6; i++) {
        if (count[i] == 2)
            temp[0] = 1; // 풀하우스(주사위 같은 수 2개 확인)
        else if (count[i] == 3)
            temp[1] = 2; // 풀하우스(주사위 같은 수 3개 확인)
        if (count[i] != 0) // temp2 =>연속된 숫자인지를 확인하는 임시
            temp[2] += 1;
        else if (count[i] == 0) //연속되는 수가 없으면 0으로 초기화
            temp[2] = 0;
    }

    if (count[i] >= 4) { // 포카드 (점수 족보 배열의 7번)
        score[7] = score[6];
    }

    if (temp[2] == 4) { // 스몰 스트레이트 (점수 족보 배열의 9번)
        score[9] = 15;
    }

    if (temp[2] == 5) { // 라지 스트레이트 (점수 족보 배열의 10번)
        score[10] = 30;
    }

    if (temp[0] == 1 && temp[1] == 2) { // 풀하우스 (점수 족보 배열의 8번)
        score[8] = score[6];
    }

    if (count[i] == 5) { // 요트 (점수 족보 배열의 11번)
        score[11] = 50;
    }
}

```

6.2 제작시 문제점 및 개선사항

- 1) gotoxy 함수 : 함수를 처음 써봐서 인터페이스를 만드는 과정 중 원하는 위치에 정확하게 출력하는 것이 어려웠다.
- 2) 굴리기 모듈 : 해당 모듈은 사용자로부터 숫자를 입력받아야하는 경우가 많은데 잘못된 숫자를 입력받았을 때 오류가 나거나 무한루프에 빠지는 등 문제가 있었지만, getchar 함수를 이용해 입력버퍼를 비우는 것으로 해결하였다.
- 3) 변수 재정의 : 최종적으로 함수들을 합치는 과정에서 자꾸 변수관련으로 문제가 많이 생겨 일단 실행이 되게 하고 싶어 메인 함수에 변수를 다 써넣었는데 여러함수에서 같은 변수를 정의해서 실행이 안되어버리는 문제가 발생했었다. 정의는 메인함수에서만 하고 나머지 함수에서는 extern으로 사용하는 것으로 해결하였다.
- 4) 함수 명 오타 : 처음에 함수가 정의되지 않았으면서 int형으로 반환한다? 라는 경고와 함께 계속 오류가 났었는데 알고 보니 소스파일 또는 헤더 파일의 함수 이름에 오타가 있어 발생한 문제였다.

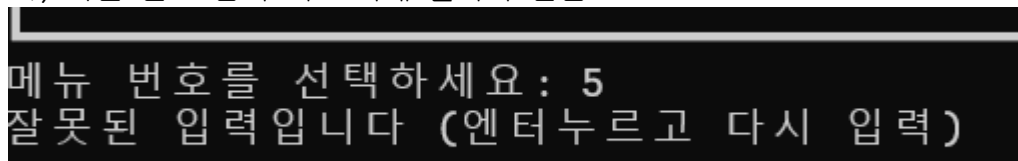
6.3 지도교수 지도내용 및 지적사항 조치결과

일자	지도내용	지적사항	조치결과	비고
2024-05-06	초기 설계 검토	순서도 형태 개선 필요	순서도의 기호와 선의 일관성과 통일성 확보	

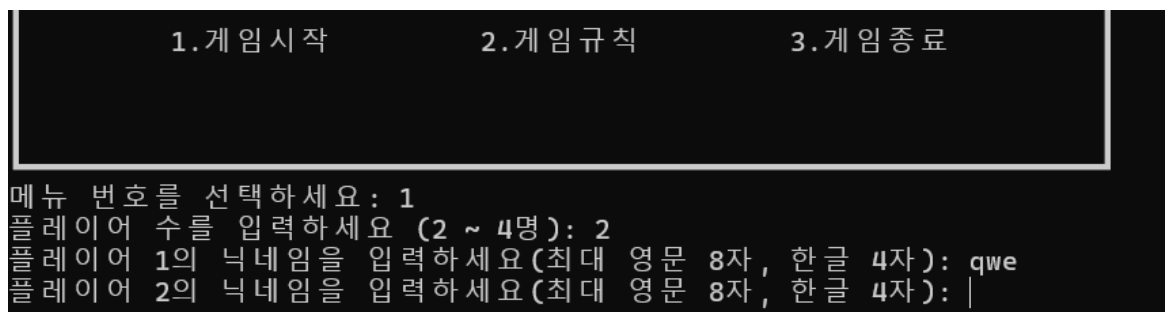
7. 시험

7.1 메뉴화면

- 1) 다른 번호 입력 시 : 기대 결과와 같음



- 2) 제대로 입력 시 : 기대 결과와 같음



7.2 규칙화면

1) 다른 번호 입력 시 : 기대 결과와 같음

```
1. 규칙 출력
2. 점수 족보 출력
3. 종료
선택 : 5
잘못된 선택입니다. 다시 선택하세요.
```

```
1. 규칙 출력
2. 점수 족보 출력
3. 종료
선택 :
```

2) 제대로 입력 시 : 기대 결과와 같음

```
Ones: 1이 나온 주사위의 합
Twos: 2가 나온 주사위의 합
Threes: 3이 나온 주사위의 합
Fours : 4가 나온 주사위의 합
Fives: 5가 나온 주사위의 합
Sixes: 6이 나온 주사위의 합
Choice: 주사위의 총합
Four of a Kind: 동일한 주사위가 4개 이상일 때, 주사위의 합
Full House: 주사위 묶음을 2,3개 또는 3,2개로 하였을 때 각 묶음에서 주사위 눈이 같을 때 주사위의 합
Small Straight: 이어지는 주사위가 4개 이상일 때 15점
Large Straight: 이어지는 주사위가 5개일 때 30점
Yacht: 주사위 눈이 전부 동일할 때 50점
```

```
1. 규칙 출력
2. 점수 족보 출력
3. 종료
선택 :
```

7.3 플레이어 정보 입력

1) 플레이어 수 잘못 입력 시 : 기대 결과와 같음

```
플레이어 수를 입력하세요 (2 ~ 4명): 1
최소 2명, 최대 4명까지 가능합니다. 다시 입력해주세요.
플레이어 수를 입력하세요 (2 ~ 4명): 5
최소 2명, 최대 4명까지 가능합니다. 다시 입력해주세요.
플레이어 수를 입력하세요 (2 ~ 4명):
```

2) 닉네임 글자 수 초과 시 : 기대 결과와 같음

```
플레이어 1의 닉네임을 입력하세요(최대 영문 8자, 한글 4자): 가나다라마
영문 8자 또는 한글 4자 이내로 입력해주세요.
플레이어 1의 닉네임을 입력하세요(최대 영문 8자, 한글 4자): qwertasdf
영문 8자 또는 한글 4자 이내로 입력해주세요.
플레이어 1의 닉네임을 입력하세요(최대 영문 8자, 한글 4자):
```


3) 닉네임에 영어와 한글 혼합사용 시 : 기대 결과와 다름

- 혼합사용을 하니 글자 수가 넘어가도 처리가 되었고, 점수판도 망가졌다. 해결 방안을 찾지 못해 암묵적으로 혼합사용은 안하기로 하였다.

category	한글	qwer	gg	lgg	l-
1. Ones	0	0	0	0	0

4) 제대로 입력 시 : 기대결과 와 같음

1. 게임 시작	2. 게임 규칙	3. 게임 종료
----------	----------	----------

메뉴 번호를 선택하세요 : 1
 플레이어 수를 입력하세요 (2 ~ 4명): 2
 플레이어 1의 닉네임을 입력하세요(최대 영문 8자, 한글 4자): qwe
 플레이어 2의 닉네임을 입력하세요(최대 영문 8자, 한글 4자): |

7.4 주사위 굴리기

1) 더 굴릴 건지 질문에 대한 입력 (Y, N이 아닌 다른 문자)

- 별 다른 출력 없이 질문 다시 출력

2) 몇 개의 주사위를 굴릴 건지 질문에 대한 입력(1~5 숫자가 아닌 다른 숫자)

- 별 다른 출력 없이 질문 다시 출력

3) 다시 굴릴 주사위 번호 입력 시 공백 없이 입력 또는 더 많은 숫자 입력 또는 1~5 숫자가 아니거나 중복일 경우 : 기대 결과와 같음

1턴, adf의 차례입니다
 잘못된 입력입니다. 2개의 유효한 번호를 입력하세요.

7.5 점수선택

1) 카테고리 잘못 입력 시 : 기대 결과와 같음

1턴, qer의 차례입니다
 1~12사이의 숫자를 입력해주세요

2) 비정상적으로 큰 숫자 입력 시 : 기대결과와 다름

- 해결방법을 찾지 못했지만 일반적으로 만 이상의 숫자를 입력할 일이 없을 것이다.

```
1턴, asdf의 차례입니다
원하는 카테고리를 입력하시오(1 ~ 12) : 125658

c:\hy\2-1\프설\비주얼스튜디오\yacht\x64\Debug\yacht.exe(프로세스 1976개)이(가) 종료되었습니다(코드 : -1073741819개).
이 창을 닫으려면 아무 키나 누르세요...
```

8. 평가

8.1 정량적/정성적 목표달성도 평가

항목	목표값	달성률 (%)	비고
프로그램의 구현	요트다이스 보드게임을 c언어로 구현	100	
인터페이스	사용자 친화적 인터페이스 구현	100	
모듈화	각 기능을 모듈화하여 유지보수가 용이한 코드 구현	90	
구현 기능	기능적 요구사항 명세서의 내용	90	

8.2 현실적 제한요소 달성도 평가

현실적 제한 요소들	목표	달성결과
경제	-게임의 구현에 필요한 메모리 등을 효율적으로 활용 -최적화된 코드 작성으로 자원의 낭비 최소화	70%
편리	- 사용자의 쉬운 게임접근을 위한 인터페이스 제공 - 명확한 게임설명 안내 제공	100%
윤리	- 사행성 목적으로 사용되지 않도록 함	100%
사회	- 여러 사람과 즐길 수 있는 게임 구현으로 화합 도모	100%

8.3 기능적 요구사항 달성도 평가

ID	요구사항	내용	우선 순위	달성률 (%)	비고
R_01	점수 계산	주사위 결과를 바탕으로 점수를 계산한다.	3	100%	
R_02	점수판 출력	입력된 결과를 점수판으로 출력한다.	5	100%	
R_03	점수 선택	자신의 주사위 결과를 자신이 원하는 점수 카테고리에 입력한다.	4	90%	
R_04	점수 추천	점수판에 주사위 결과를 바탕으로 선택하기 좋은 카테고리들을 표시한다.	10	100%	
R_05	다중 플레이어 구현	다중 플레이어 게임을 구현한다.	6	95%	
R_06	주사위 굴리기	5개의 주사위를 굴린다.	1	100%	
R_07	규칙 설명	사용자에게 게임 시작 전 게임에 대한 규칙을 제공한다.	9	100%	
R_08	승패 여부	다중 플레이어간 승패여부를 확인한다	11	100%	
R_09	턴 변경	플레이어가 행동을 마치면 턴을 변경한다.	7	100%	
R_10	게임 메뉴 인터페이스	게임 메뉴 인터페이스를 제공한다.	8	100%	
R_11	주사위 정렬	주사위를 오름차순으로 정렬한다	2	100%	

8.2 문제점 및 해결방안

- 1) 경제성 관련 : 각자 함수를 구현할 때 생각하지 못했던 문제들이 합쳐서 실행시켰을 때 많이 발생하였다. 예를 들면, 이미 입력한 점수에 점수가 또 입력이 가능하던가, 점수 계산 시에 이전 주사위 결과값까지 점수 계산 시에 합산이 된다던가 하는 문제가 있었다. 최우선 목적을 실행 가능 여부로 잡았기에 합쳐서 실행이 되고 나서는 크게 뜯어고칠 엄두가 나지 않아 이렇게 하나하나 발견되는 문제를 해결하기 위해 그때그때 변수를 또 새로 만들어서 함수를 덧붙여 해결하였기에 메모리나 시간복잡도 등을 크게 고려하지 못했다.
- 2) 점수선택함수 : 점수 선택 함수에서 카테고리를 고르는 상황에서 1에서 12 사이의 숫자만 골라야하는데 천 정도의 단위까지는 함수대로 잘못 입력하였다는 메시지가 출력되는데 십만, 백만 자리로 넘어가면 그냥 코드가 종료되어버린다. 함수 자체에는 문제가 없는 것으로 보여 원인을 알아내지 못했다.
- 3) 플레이어 이름 : 처음에 이차원 배열을 사용해서 글자 수 제한을 정해놨었는데 초과해도 오류메세지를 출력하지 않아서 아스키코드를 사용하여 한글과 영어를 나누어 한글4자 영어8자로 8바이트까지 입력 가능하게 하고 초과될 시 오류메세지를 출력하게 하였다. 하지만 한글과 영어를 혼합하여 이름 설정 시 처음 원하는대로 초과하는 것을 잘 잡지 못해 8바이트가 넘어가게 되었다. 해결방법을 찾지 못하였고 처음 함수를 구현할 때도 한글과 영어는 혼합해서 사용하는 경우를 생각 안하였기에 암묵적으로 한글과 영어를 따로 사용하기로 하였다.

9. 추진체계

본 과제의 추진을 위한 팀은 4인으로 구성되며 다음과 같은 역할을 담당한다. 이론 연구 단계에서부터 과제 종료시점까지 협력과 보완을 통해 팀의 목표를 달성할 수 있도록 한다.

조장 (이현영) : 각 기능에 맞는 사용자 정의함수 작성, 과제총괄 및 보고서 작성

팀원 (강현욱) : 각 기능에 맞는 사용자 정의함수 작성, 게임의 주요 알고리즘 구현

팀원 (김승호) : 각 기능에 맞는 사용자 정의함수 작성, 코드 최적화

팀원 (이윤상) : 각 기능에 맞는 사용자 정의함수 작성, 테스트 및 디버깅

10. 설계 추진 일정: 2024년 4월 ~ 2024년 6월

수행 내용		일정 (2주 단위)							
		1,2	3,4	5,6	7,8	9,10	11,12	13,14	15,16
목표와 기준 설정	- 설계목표 설정 - 시장조사, 문헌조사								
	- 소프트웨어 요구사항 파악								
합성	- 기능 블록 구성 - 기능별 구현방법 결정 - 적용할 이론 및 기술 - UI 설계								
분석	- 세부 기능 블록도 - 목표달성가능성 확인 - Pseudo Cod술 또는 순서도 작성								
제작	- 프로그램 coding - UI 구현								
시험/ 평가	- 시험 및 검증 - Trouble shooting - 재설계								
결과	- 결과보고서 작성								

11. 결론

- 최종적으로 목표로 정했던 것은 다 해내었다고 생각한다. 하지만 시험 및 검증 단계에서 기대결과와 다른 부분들과 오류가 나는 부분들을 해결하기 위해 자원의 낭비나 함수의 시간복잡도까지 전부 고려하지 못하였던 부분이 아쉽다.

본 설계과정을 통해 프로그래밍 능력을 향상시킬 수 있었다. 함수 하나하나를 개별적으로 배우는 이론적인 것과 전체적인 것을 다 고려하며 함수를 구현해야하는 실습은 천지차이라는 것을 느꼈고, 질 좋은 경험이 되었다고 생각한다. 또한 이 과정에서 설계부터 구현까지의 그 과정을 두 달에 가까운 시간동안 팀프로젝트를 통해 팀원들과 해결해나가면서 여러 가지를 배우고 느낄 수 있었다. 의사소통 능력을 향상시킬 수 있었고, 각자의 역할을 조율하여 책임지고 목표를 달성해가는 것의 중요성을 알았고, 팀원들로부터 다양한 관점을 접하게 돼 새로운 시각으로 문제를 바라볼 수 있게 되었다. 개인의 역량만이 아니라 팀으로서의 능력을 향상시키는 경험이 되었다.

부록

<프로그램 소스코드>

- 라인별 커멘트 작성
- 사이버캠퍼스에 업로드
- 안되면 대용량첨부로 이메일로 전송